C Programming

Pointers: Ch.11

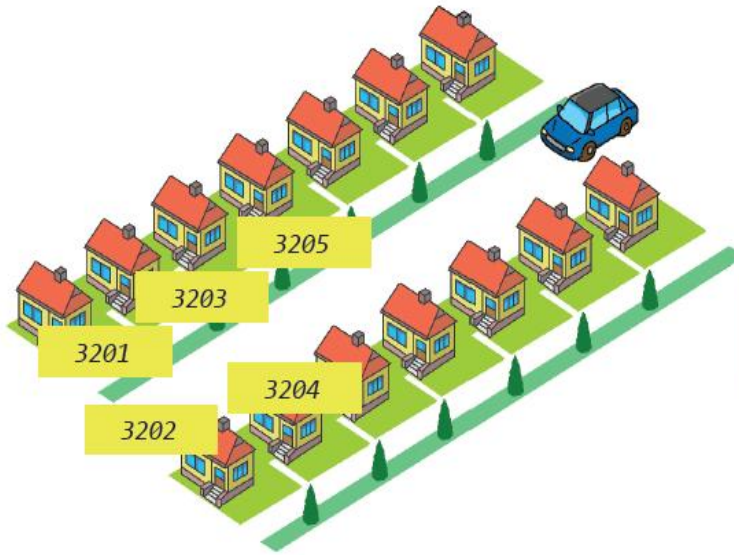
Addresses | Declaring & dereferencing | Call by reference | Pointers & arrays

What You Will Learn in This Chapter

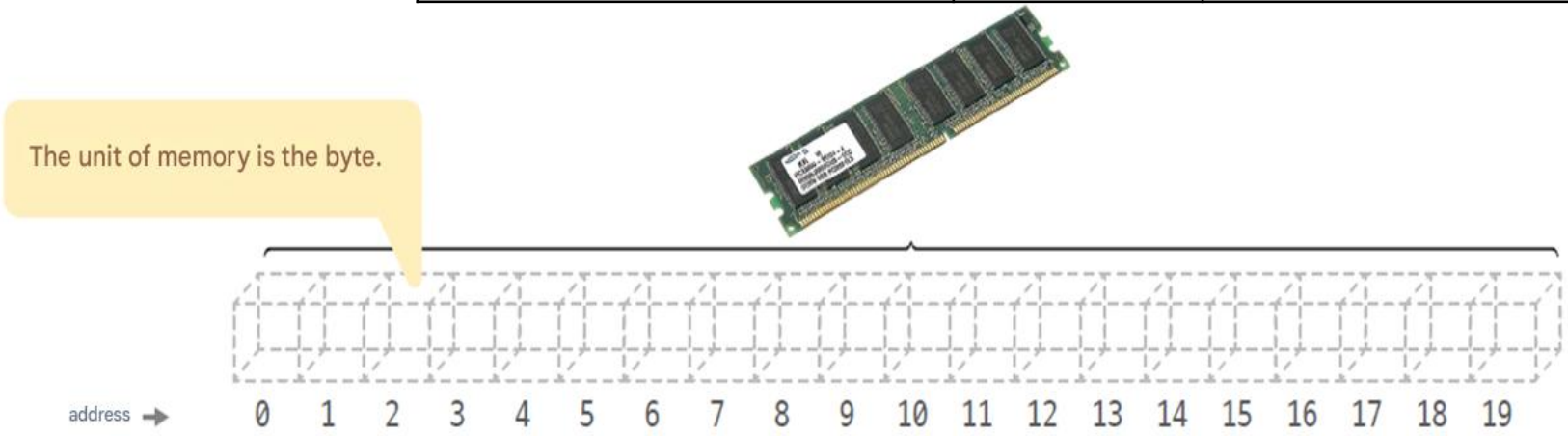
- What is a **pointer**?
  - A pointer is a **variable** that stores the **memory address of another variable**.
- **Address** of a variable
  - Use the **& operator** to get a variable's address.
- Declaration of a **Pointer**
  - ***** in a declaration to create a pointer variable.
- **Indirect reference operator (Dereference Operator)**
  - The *** operator** accesses the **value stored at the address** held by a pointer.
- **Pointer arithmetic**
  - Adding or subtracting from a pointer moves it by the **size of the data type it points to**.
- Pointers and arrays, pointers and functions
  - Array names are closely related to pointers, and pointers allow functions to **access or modify original data through addresses**.

What Is a Pointer?

- **Pointer**: A variable that has an **address**.
- Variables are stored in memory. Memory is accessed in bytes (the minimum unit).
- Size of **pointer**: the size of a **pointer** depends on the SYSTEM (architecture), not the type of variable it points to. sizeof(pi) == sizeof(pc) == sizeof(pd) - **always the same.**



System / Architecture	Address size	Pointer size
32-bit systems (x86)	32 bits	4 bytes
Windows 64-bit (x86-64, ARM64)	64 bits	8 bytes
Linux/macOS 64-bit (LP64)	64 bits	8 bytes



Variable & Memory

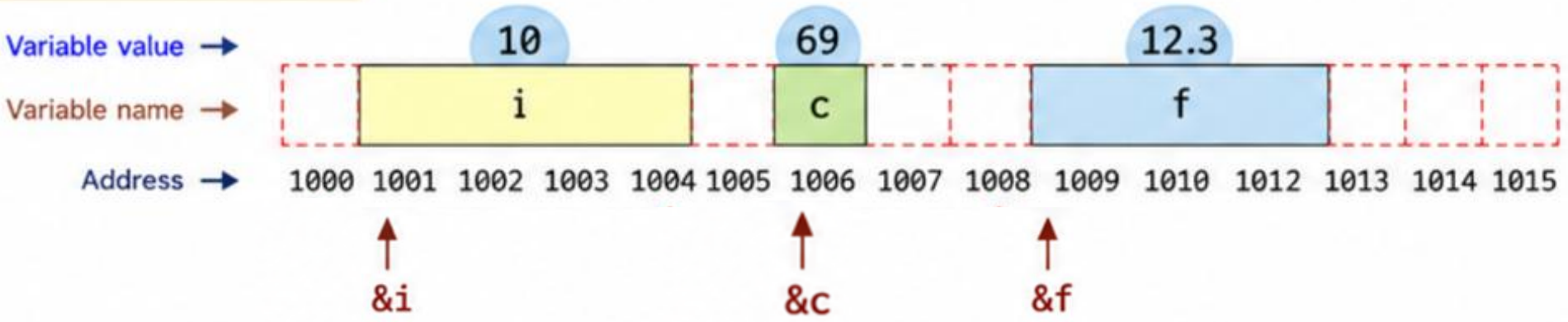
VSC

- The memory space occupied varies depending on the size of the variable
- & Operator used to obtain the address of a variable.

Variables and Their Addresses

```
int main( void )
{
    int i = 10;
    char c = 69;
    float f = 12.3;
    return 0;
}
```

Variable	Value	Address	Type	Size
i	10	**1001**	int	4 bytes
c	69	**1006**	char	1 byte
f	12.3	**1009**	float	4 bytes



Address of a Variable: the & Operator

```c
int main( void )
{
    int i = 10;
    char c = 69;
    float f = 12.3;

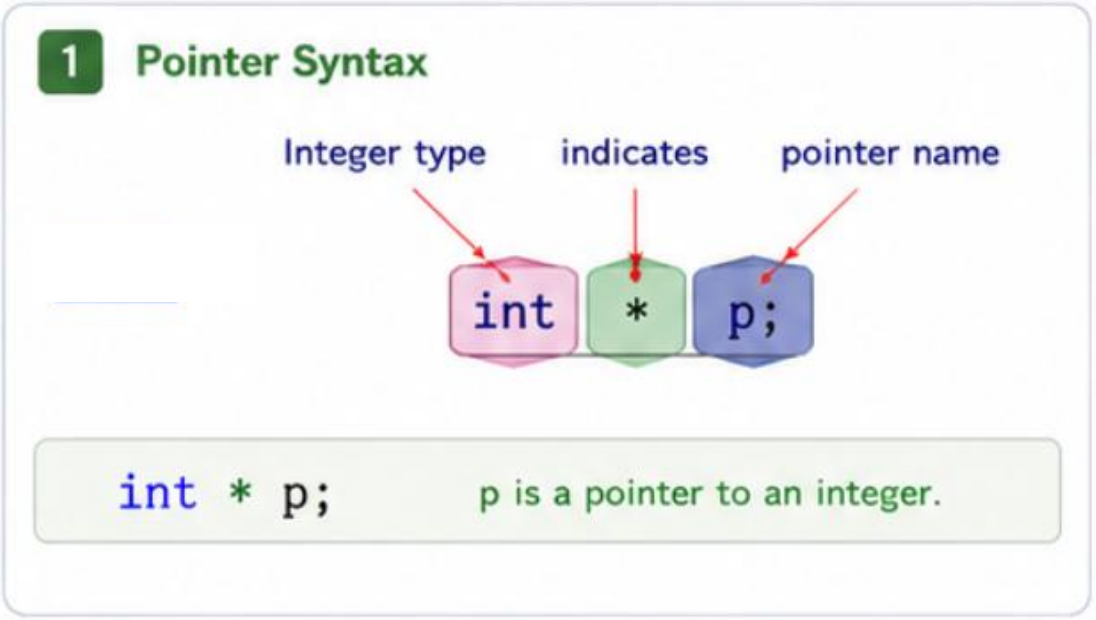
    printf ( "Address of i: %p\n" , &i ); // address of i
    printf ( "Address of c: %p\n" , &c);   // address of c
    printf ( "Address of f: %p\n" , &f);   // address of f
    return 0;
}
```

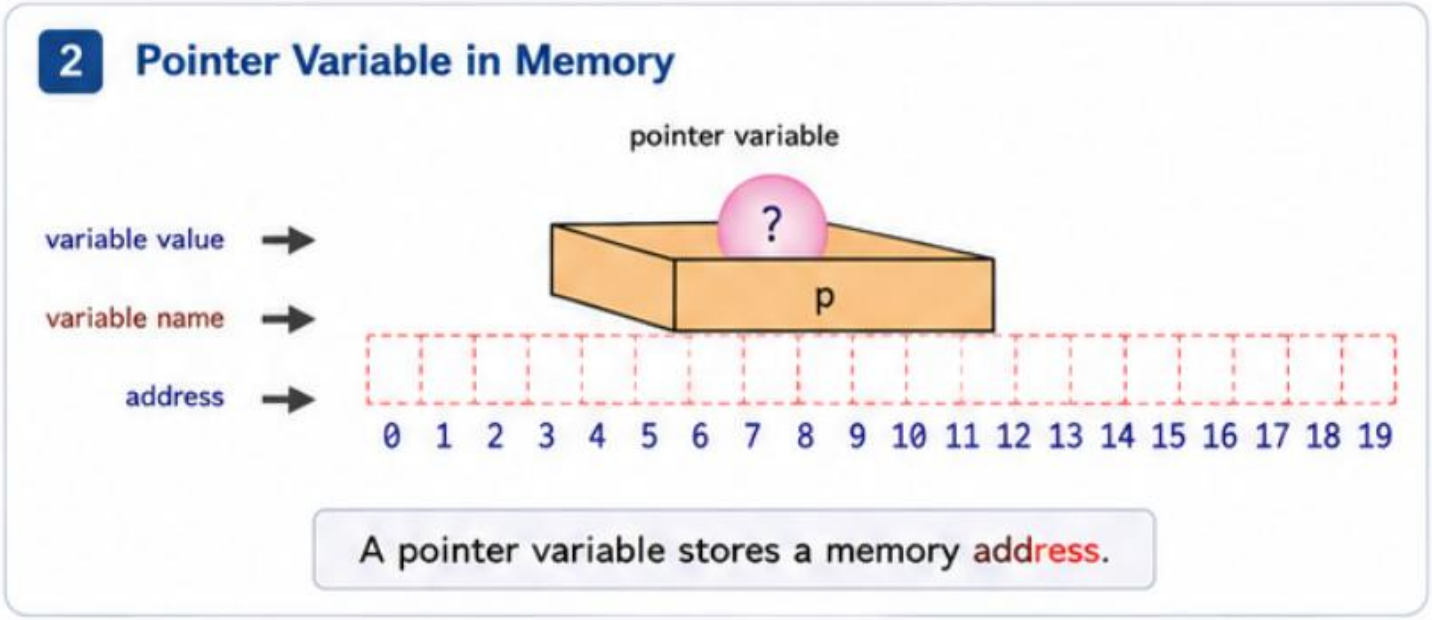
```
i 's address : 0000003D69DDF974
Address of c : 0000003D69DDF994
Address of f : 0000003D69DDF9B8
```

The program's addresses will be different each time you run it.

Declaration of a Pointer

- A pointer is declared by specifying the data type it points to, followed by an asterisk (*), and then the pointer's name





Assign the address of variable to Pointers vs. Access the value at the address

```
int i = 10;

int *p;
p = &i;

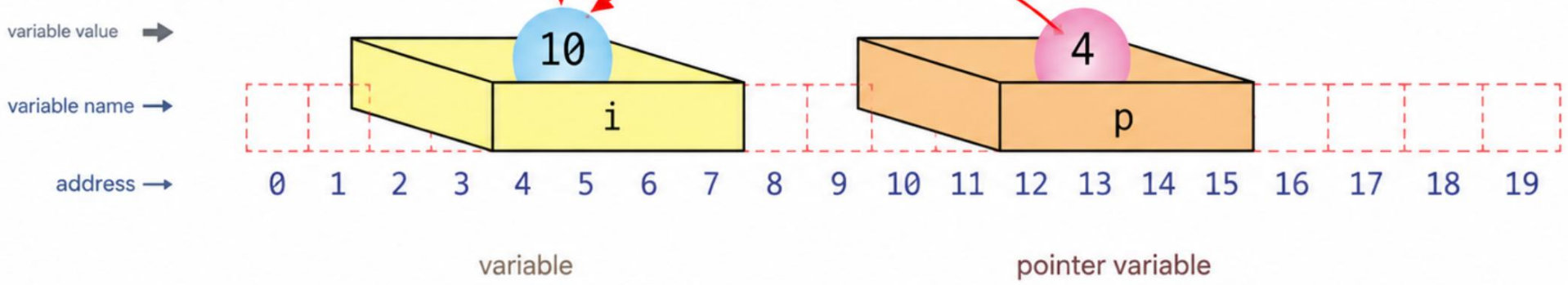
printf( "%d \n", *p);
```

A pointer variable is a variable that stores an address,

The dereference (indirection) operator is used to access the value stored at that address.

*p

* Returns the value of variable pointed by the pointer



Two Roles of "*" in C

1 Pointer Declaration

*** in a declaration means "pointer to".**

```
int   *p;    //  p is a pointer to int
```

type — pointer symbol — pointer variable

Example

```
int i = 10;
int *p;
p = &i;      // &i : address of i is stored in p
             // &i is the address of i
```

& (address-of operator)

Gets the address of a variable.

2 Dereference Operator

*** in an expression means "value at the address".**

```
*p    // value at the address stored in p
```

Reads (or writes) the value at the address held by pointer p.

Example

```
int i = 10;
int *p = &i;
printf("%d\n", *p);  // Output: 10
```

*** (dereference operator)**

Gets the value at the address.

NULL Pointer: Initialization of the Pointers

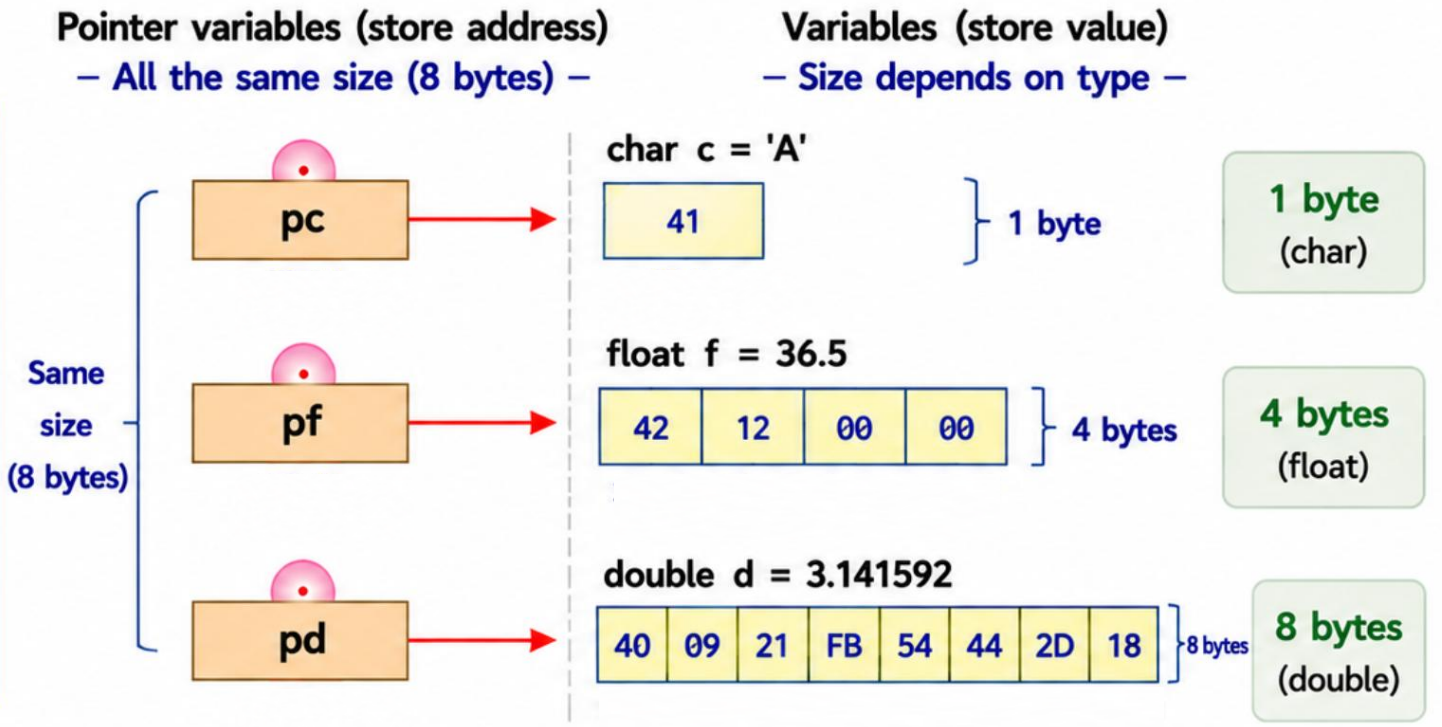
- NULL is defined in stdio.h. It represents **address** 0,
  - defined as: #define NULL ((void *)0)
- **Address** 0 is generally unusable (the CPU reserves it for interrupts).
  - So if a **pointer**'s value is 0, we can assume it is not pointing to anything.
- int *pi = NULL; double *pf = NULL;
  - a NULL **pointer**, pointing to nothing.

- You should not use uninitialized pointers.
  - The **pointer** contains an arbitrary garbage **address**, so we have no way of knowing what it is pointing to.
- If a **pointer** points to nothing, initialize it to NULL:
  - int *p = NULL;
- The type of the **pointer** and the type of the variable must match
  - double *pd = &i; (where i is int) is an error.

Pointers to Difference Data Types & NULL Pointer

```
char   c = 'A';        // character variable c
float  f = 36.5;       // real number variable f
double d = 3.141592;   // real number variable d

char   *pc = &c;       // pointer to char
float  *pf = &f;       // pointer to float
double *pd = &d;       // pointer to double
```



On a 64-bit system, every pointer variable is 8 bytes (64 bits).

= 1 byte of memory

Numbers below are memory addresses in hexadecimal (base 16).

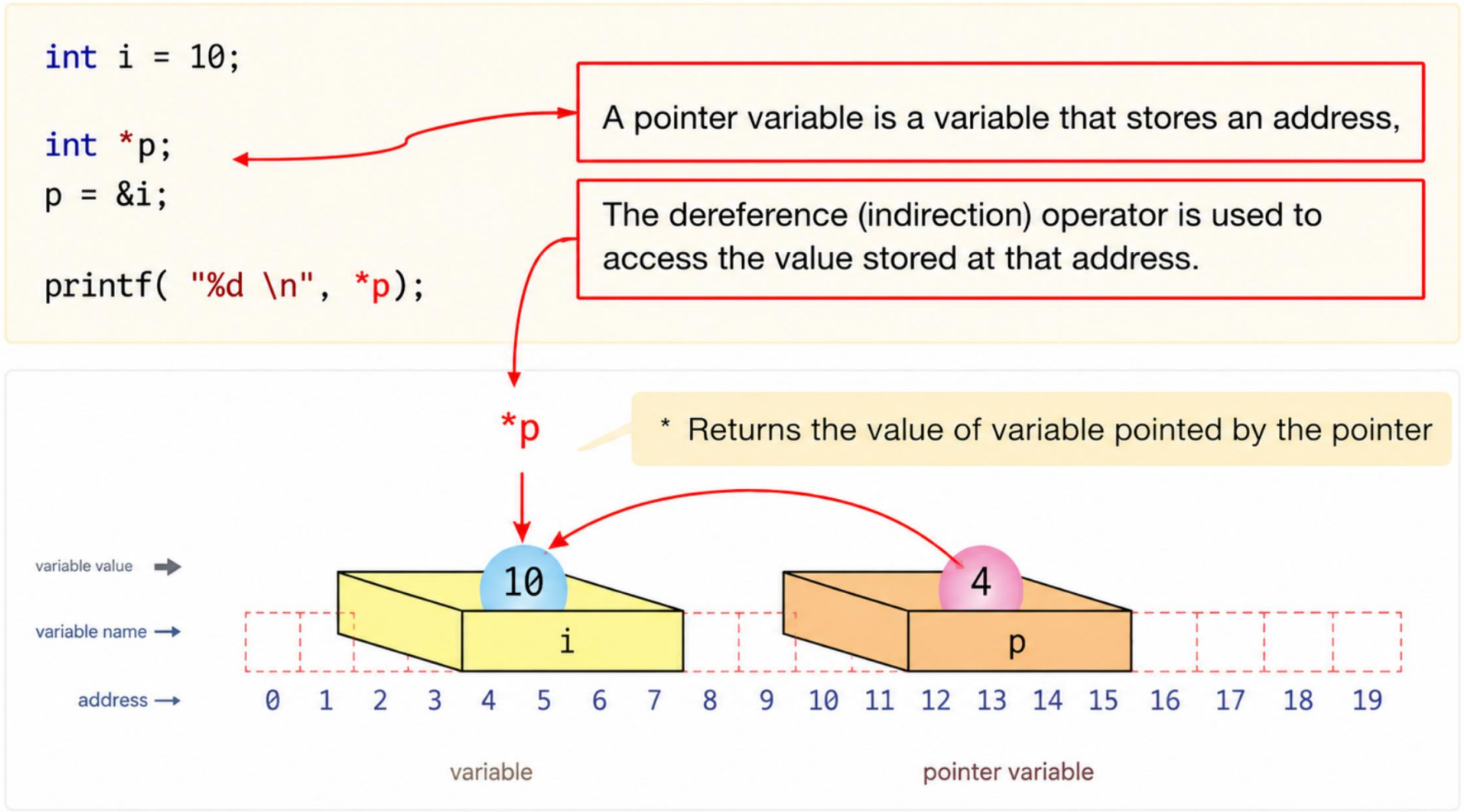
NULL pointer → **Pointing to nothing** (does not point to any valid memory address)

```
int *p = NULL;   // or int *p = 0;
```

- A NULL pointer does not point to any valid memory.
- Always check for NULL before dereferencing.
- Useful for initialization and error checking.

Indirect Reference(Dereference) Operator

- Indirect reference operator : Reads **a value** based on the type of the pointer at the specified location.



& Operator and * Operator

& (Address-of Operator)

Gets the address of a variable.

* (Indirect Reference / Dereference Operator)

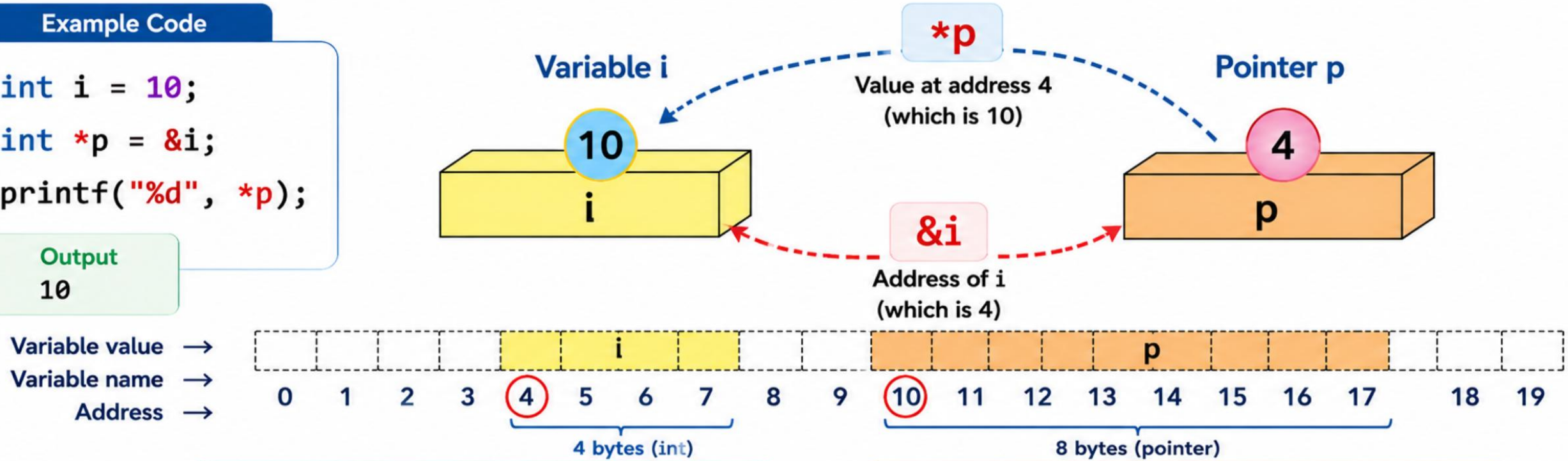
Accesses the value stored at the address held by a pointer.

Example Code

```
int i = 10;
int *p = &i;
printf("%d", *p);
```

Output

```
10
```



&i — **Address-of Operator**
Returns the memory address of variable i (which is 4).

***p** — **Dereference Operator**
Accesses the value stored at the address held by pointer p (which is 10).

Pointer Example: & Operator and * Operator

```
int i = 10;
int *p;                  // Declaration of Pointer variable
p = &i;                  // Assign the Address of the variable

printf( "%d \n" , *p);   // Dereference Operator: returns the value pointed to by p

// & operator: returns the address of a variable
// * operator: returns the contents of the location pointed to by the pointer
```

A pointer variable stores an address; the dereference operator is used to access the value stored at that address.

Pointer Example: Changing a Value Through a Pointer

```c
#include <stdio.h>
int main( void )
{
    int i =10;
    int *p;

    p = &i;
    printf( "i = %d\n" , i );

    *p = 20;
    printf( "i = %d\n" , i );   // i is now 20!
    return 0;
}
```

Pointer Arithmetic

- Possible operations: increment, decrement, addition, subtraction.
- In an increment operation, the value increased is the SIZE of the object pointed to by the **pointer**.

Data Type	Size in Bytes (Increases by ++ operation)
char	1
short	2
int	4
float	4
double	8

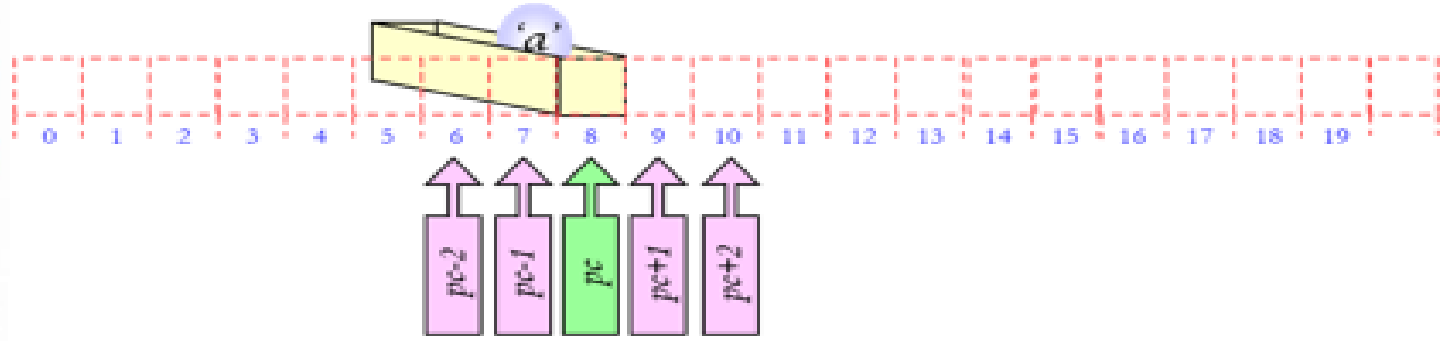
```
Char   *pc; // pc + 1 moves by sizeof(char)
int    *pi; // pi + 1 moves by sizeof(int)
double *pd; // pd + 1 moves by sizeof(double)

printf("pc=%p, pc+1=%p\n", pc, pc+1);
printf("pi=%p, pi+1=%p\n", pi, pi+1);
printf("pd=%p, pd+1=%p\n", pd, pd+1);
```

pc=10000, pc+1=10001, pc+2= 10002
pi=10000, pi+1=10004, pi+2= 10008
pd=10000, pd+1=10008, pd+2=10016

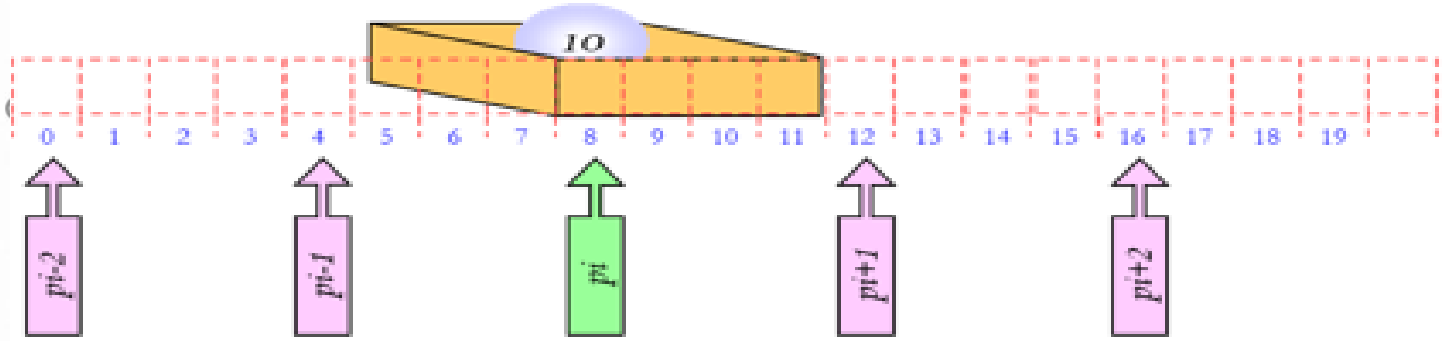
Increment and decrement operations of pointers

1) char a



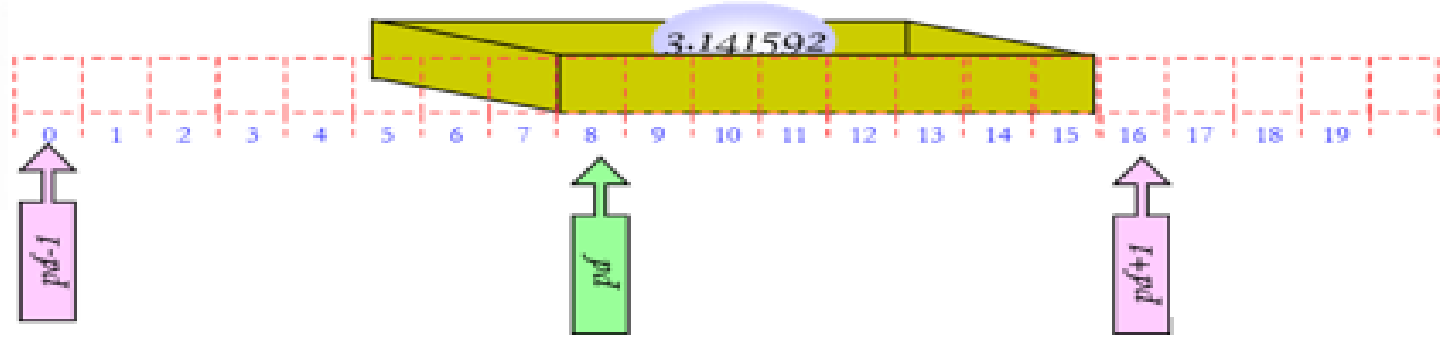
char a
Size: 1 byte
Stored at address 8

2) int i



int i
Size: 4 bytes
Stored at addresses 8~11

3) double d



double d
Size: 8 bytes
Stored at addresses 8~15

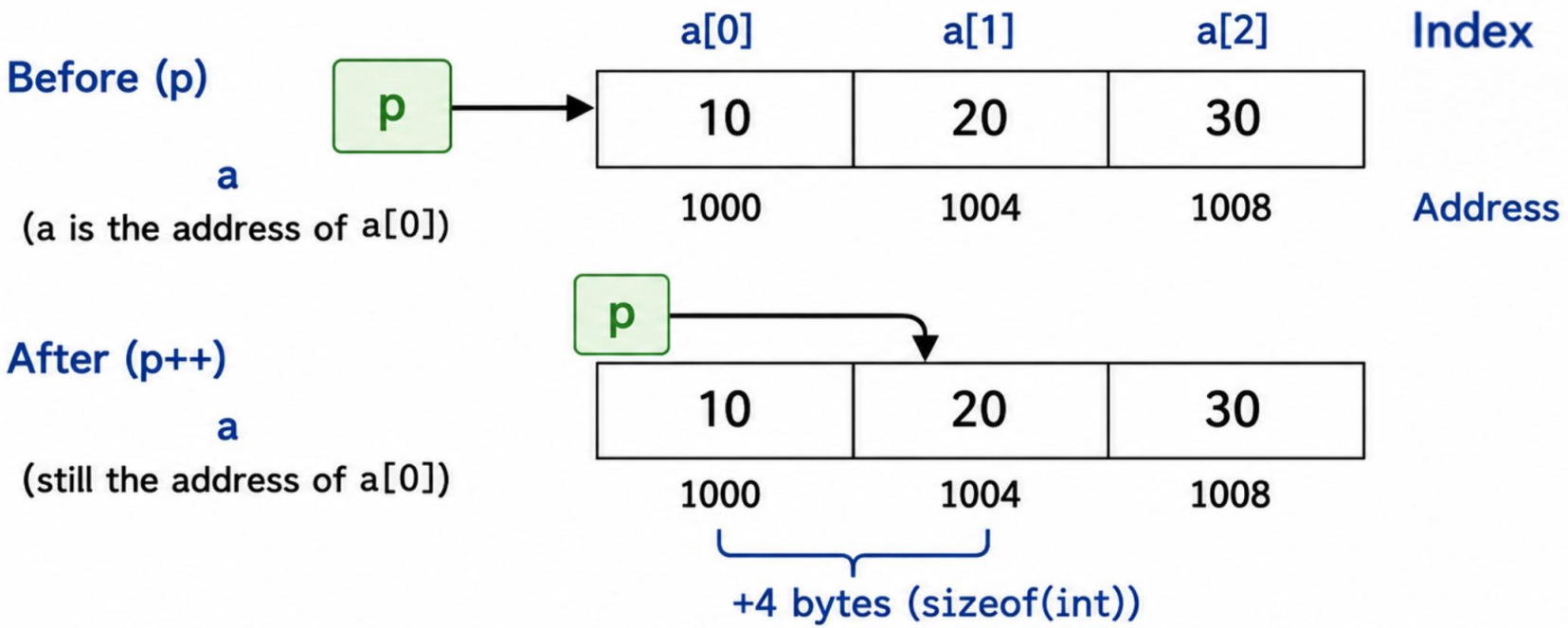
Pointer vs. Array

- An **array name** **represents the** **address of its first element**.
  When the pointer is incremented, it moves to the **next element according to the data type size**.

```
int a[] = {10, 20, 30};
```

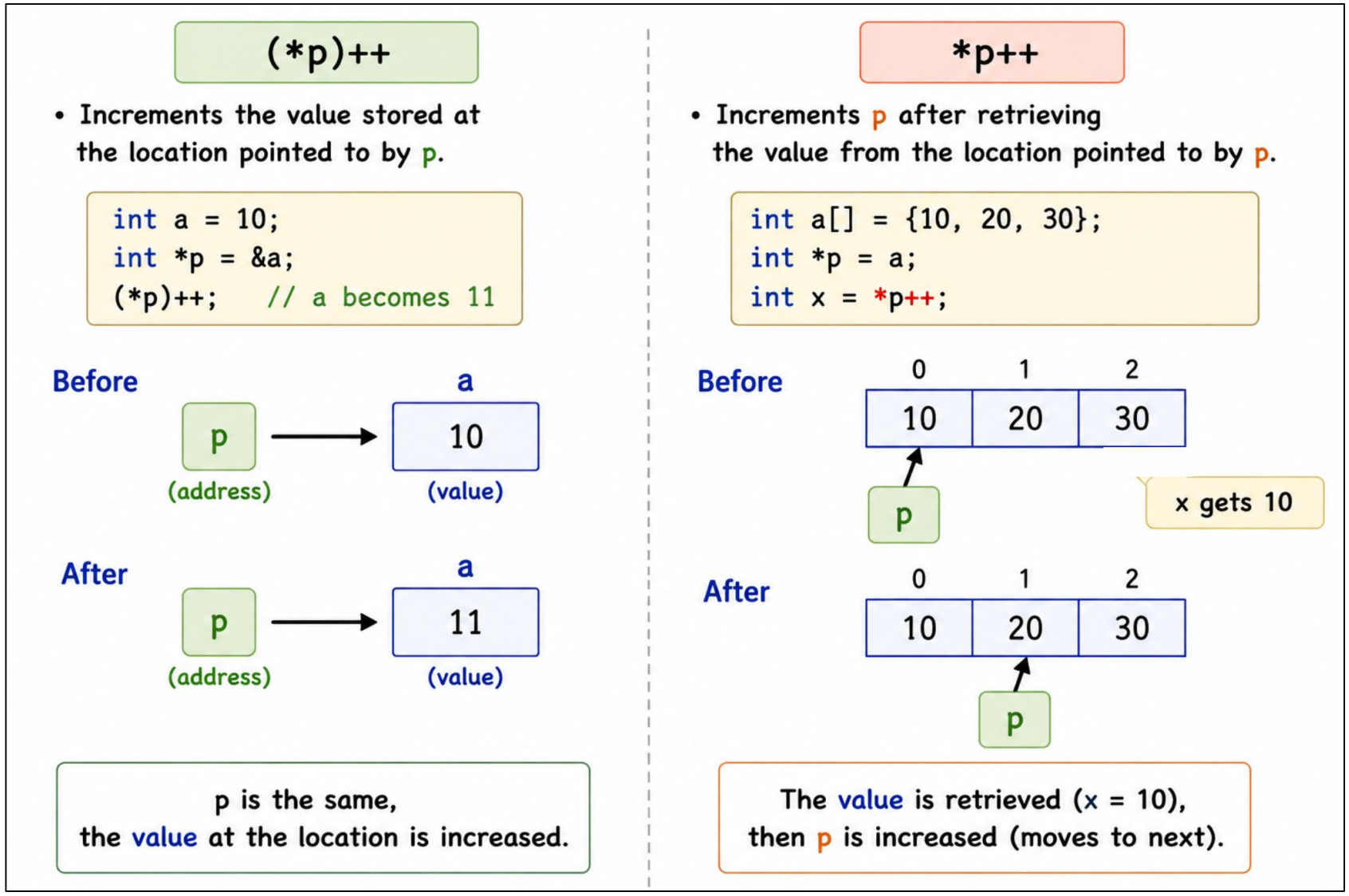
int *p = a;

- Array name (a) is the address of the first element (a[0]).
- When the pointer is incremented (p++), it moves to the next element according to the data type size (int: 4 bytes).



Pointer Increment: (*p)++ vs. *p++

- (*p)++;
  - Increments the value stored at the location pointed to by p.
- *p++;
  - Uses the value from the location pointed to by p FIRST, then Increments p afterward.



Summary: Pointer

1D Array → Pointer

- The array name represents the address of its first element.
- When passed to a function, the array decays to a pointer. **(decay = convert)**
- `int arr[]` and `int *arr` are equivalent in function parameters.

```
void func(int arr[], int size);   // same as: void func(int *arr, int size);

int main(void)
{
    int a[5] = { 10, 20, 30, 40, 50 };
    func(a, 5);          // 'a' is the address of a[0] (type int *)
}
```

Memory View of a 1D Array

Index	0	1	2	3	4
a	10	20	30	40	50
Address	1000	1004	1008	1012	1016

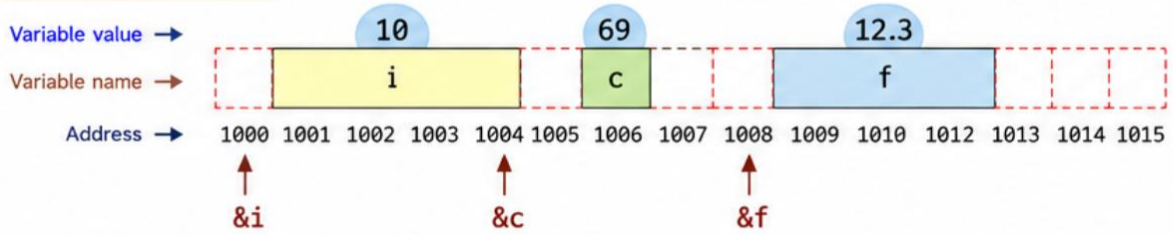
(assume int = 4 bytes)

a (array name) = address of a[0] = 1000 (type int *)

Variables and Their Addresses

```
int main( void )
{
    int i = 10;
    char c = 69;
    float f = 12.3;
    return 0;
}
```

Variable	Value	Address	Type	Size
i	10	1000	int	4 bytes
c	69	1004	char	1 byte
f	12.3	1008	float	4 bytes



Key Points

- The unit of memory is the byte.
- A variable has a name, a value, and an address.
- The address-of operator (&) gives the address of a variable.
- A pointer stores a memory address.
- The dereference operator (*) accesses the value at that address.

Examples

- `&i` → address of i (1000)
- `int *p = &i;` → p stores 1000
- `*p` → value at address 1000 (which is 10)

Remember: Arrays and variables share the same idea — both are stored in memory. Pointers let us access and manipulate that memory.

Array name represents address of the first element

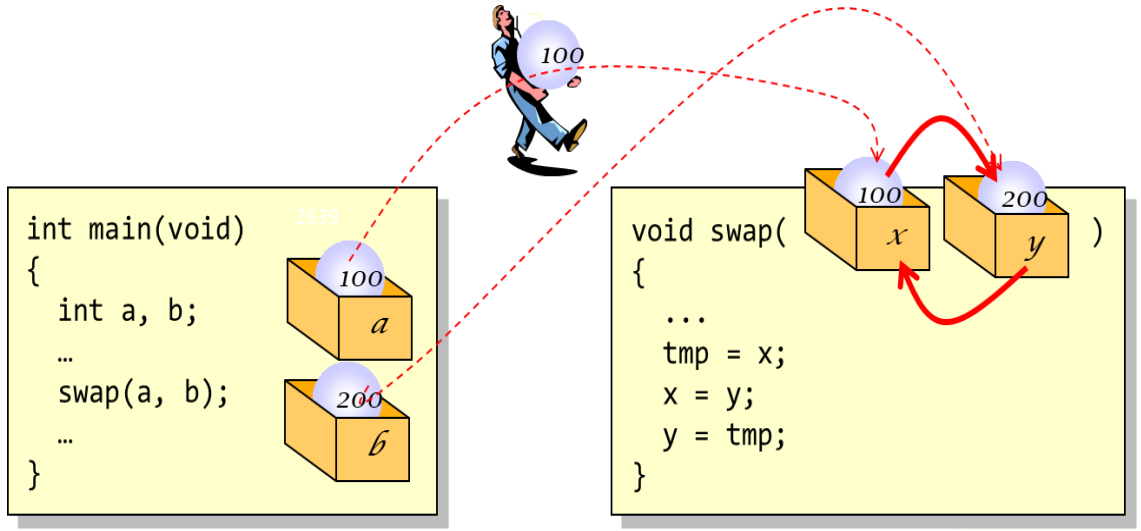
Pointer stores an address and points to data in memory

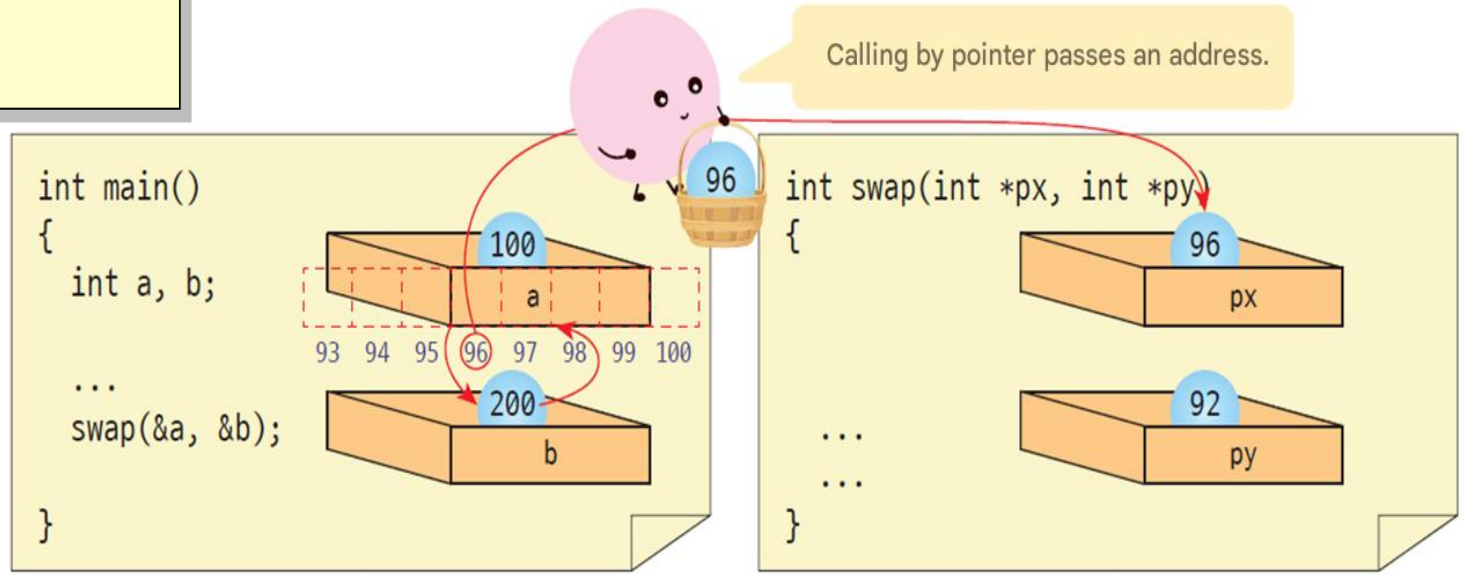
How to pass arguments to parameters

Pointers and Functions

How to pass arguments to parameters: Call by Value vs. Call by Reference

- Call by value: a copy is passed to the function - the basic method in C.
- Call by reference: the original is passed to the function - in C, this is emulated using pointers.





swap() with Call by Value (Doesn't Work)

```c
#include <stdio.h>
void swap( int x, int y);
int main ( void )
{
    int a = 100, b = 200;
    printf("a=%d b=%d\n", a, b);

    swap(a, b);

    printf("a=%d b=%d\n", a, b);
    return 0;
}
```

```
a=100 b=200
x=100 y=200
x=200 y=100
a=100 b=200
```

```c
void swap( int x, int y)
{
    int tmp ;
    printf("x=%d y=%d\n", x, y);

    tmp = x;
    x = y;
    y = tmp ;

    printf("x=%d y=%d\n", x, y);
}
// a and b in main() are UNCHANGED after
swap(a, b);
// because x and y are only local COPIES of
a and b.
```

swap() with Call by Reference (Pointers)

```c
#include <stdio.h>
void swap( int x, int y);
int main ( void )
{
    int a = 100, b = 200;
    printf("a=%d b=%d\n", a, b);

    swap(&a, &b);

    printf("a=%d b=%d\n", a, b);
    return 0;
}
```

```
a=100 b=200
x=100 y=200
x=200 y=100
a=200 b=100
```

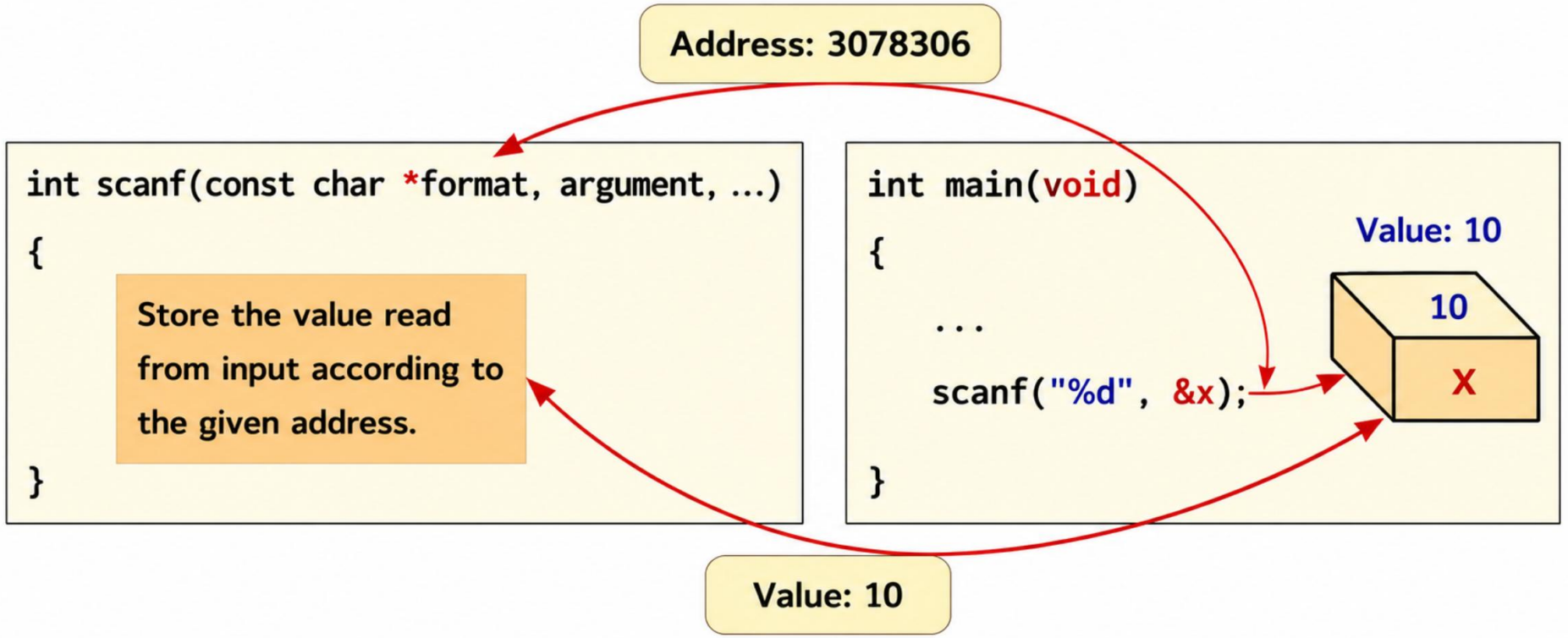
```c
void swap( int *px, int *py)
{
    printf("x=%d y=%d\n", *px, *py);

    int tmp = *px;
    *px = *py;
    *py = tmp;

    printf("x=%d y=%d\n", *px, *py);
}
// call:  swap(&a, &b);
// Now a and b in main() ARE swapped, because we
passed their ADDRESSES, not copies of their values.
```

scanf() function: Why using &

- Receives the address of a variable to store a value



&x : address of variable x

scanf("%d", &x) : reads an integer from input and stores it at the address of x

Value 10 is stored at Address 3078306 (address of x)

Returning More Than One Result

Call by Reference

- If a function needs to return more than one value, one way is to use pointers
  - Write the function so it returns the result as a parameter.
- Check the **pointer** isn't NULL before writing through it.

```c
int add_numbers(int a, int b, int *result)
{
    if (result == NULL) return -1;
    *result = a + b;
    return 0;
}
// call: add_numbers(3, 5, &sum);
```

A very close relationship

Pointers and Arrays

Pointers and Arrays

- **Arrays** and **pointers** have a very close relationship.
- The **array name** is actually a **pointer**.
- **Pointers** can be used like **arrays**
- &a[0], &a[1], &a[2] are consecutive addresses; array name equals &a[0].

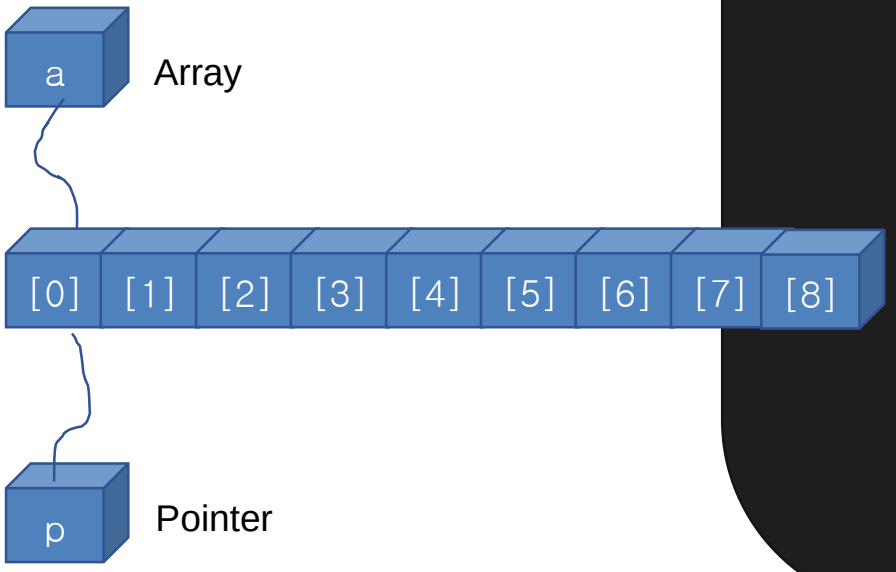
```
int a[] = { 10, 20, 30, 40, 50 };

printf( "&a[0] = %p\n" , &a[0]);
printf( "a = %p\n" , a);           // same as &a[0]

printf("*a = %d\n", *a);           // 10, same as a[0]
printf("*(a+1) = %d\n", *(a+1));   // 20, same as a[1]
```



Array Pointer

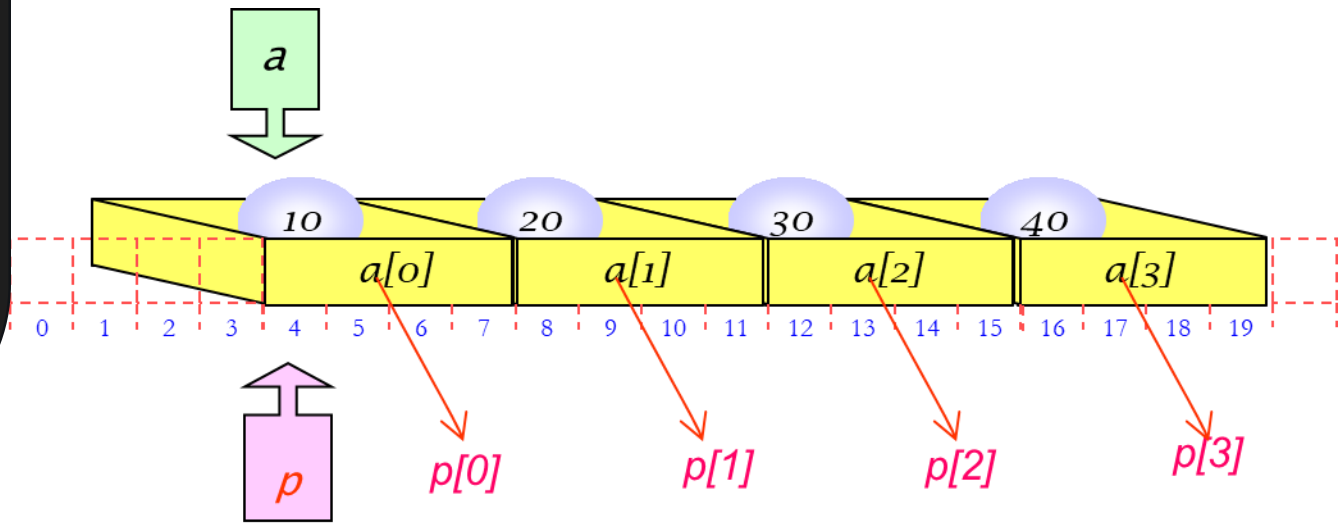


Using Pointers Like Arrays

```
int a[] = { 10, 20, 30, 40, 50 };
int *p;

p = a;
printf ( "a[0]=%d a[1]=%d a[2]=%d\n" ,
a[0], a[1], a[2]);
printf ( "p[0]=%d p[1]=%d p[2]=%d\n" ,
p[0], p[1], p[2]);

p[0] = 60;
p[1] = 70;
// a[0] and a[1] change too - arrays are
ultimately implemented as pointers!
```



Index notation can be used with pointers, just like with arrays.

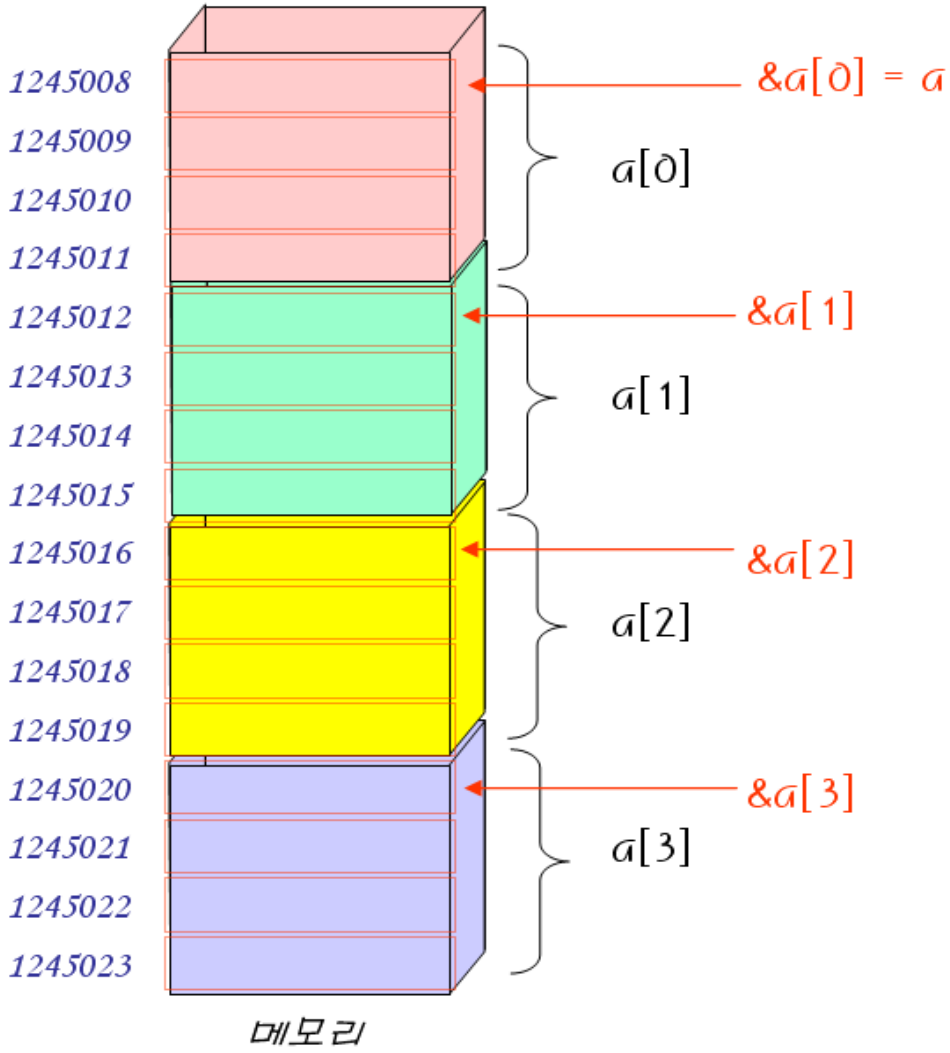
Pointers and Arrays

```c
#include <stdio.h>
int main ( void )
{
    int a[] = { 10, 20, 30, 40, 50 };

    printf( "&a[0] = %p\n" , &a[0] );
    printf( "&a[1] = %p\n" , &a[1] );
    printf( "&a[2] = %p\n" , &a[2] );

    printf( "a = %p\n" , a );
    return 0;
}
```

```
&a[0] = 1245008
&a[1] = 1245012
&a[2] = 1245016
a = 1245008
```



Index notation can be used with pointers, just like with arrays.

Array & Pointer: Value and Address Access

- Pointers can be used like arrays.
- Index notation can be used with pointers.

What about *(&arr[0])?

```
int a[5] = {10, 20, 30, 40};
int *p;     ← How to declare the pointer variable
p = a;
```

[Array] [Pointer]

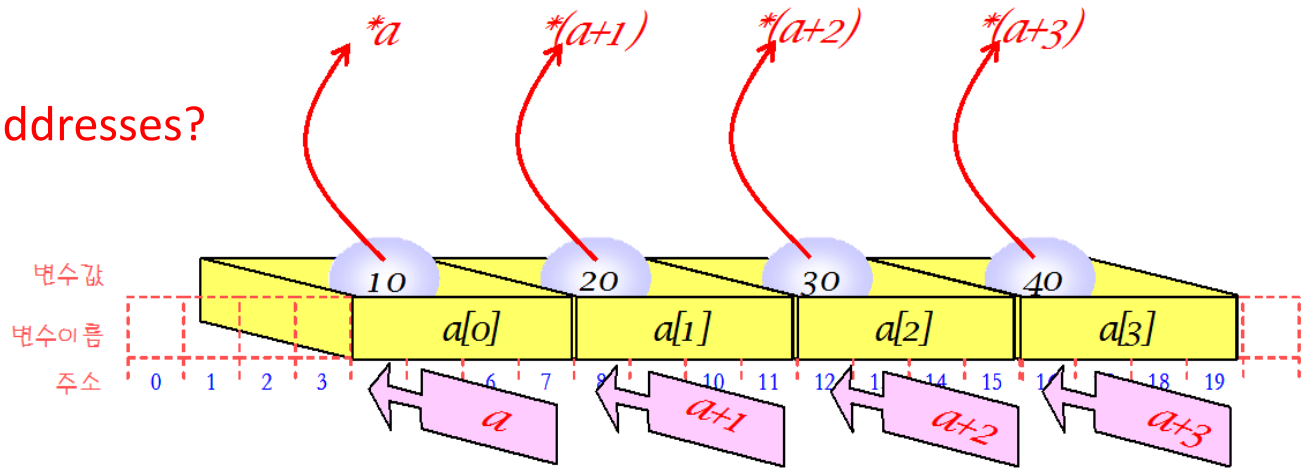
a[0] == *(p + 0) == *p ← How to access the values?

a[1] == *(p + 1)

—----------

&a[0] == (p + 0) == p == a ← How to access the addresses?

&a[1] == (p + 1)



Array & Pointer: Value and Address Access

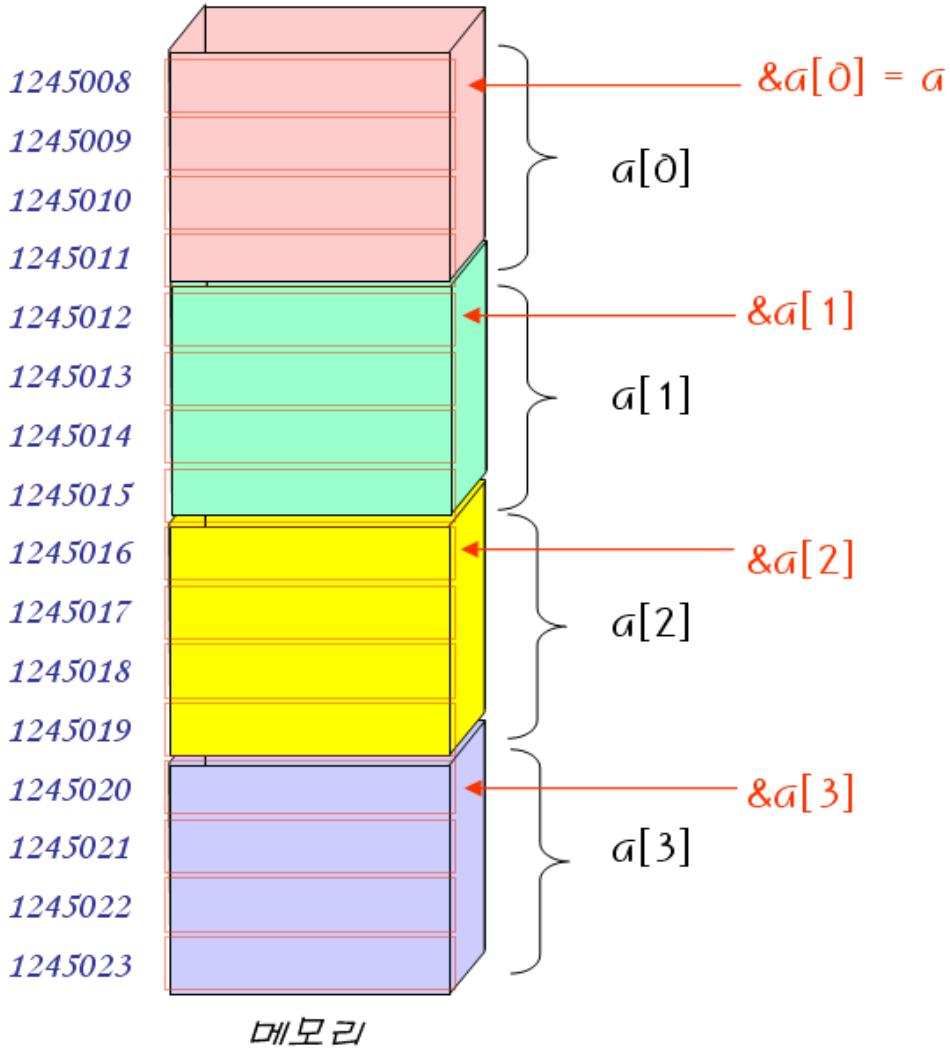
```c
#include <stdio.h>
int main ( void )
{
    int a[] = { 10, 20, 30, 40, 50 };

    printf( "a = %p\n" , a );
    printf( "a+1 = %p\n" , a+1 );

    printf( "*a = %d\n", *a );
    printf( "*(a+1) = %p\n" , *(a+1));
    return 0;
}
```

```
a = 1245008
a + 1 = 1245012
*a = 10
*(a+1) = 20
```

Index notation can be used with pointers, just like with arrays.



Array Parameters

- General parameters vs. array parameters.

```
// Parameters x at memory place
void sub( int x)
{
…
}
```

```
// Parameters b does not have memory allocated.
void sub( int b[] )
{
…
}
```

- Why? Copying an array to a function would be time-consuming, so C only passes the **ADDRESS** of the array.

- Array parameters can be thought of as pointers.

```
int main(void)
{
        int a[3]={ 1, 2, 3 };

        sub(a, 3);
}
```

The name of the array is a pointer.

```
void sub(int b[], int size)
{
  b[0] = 4;
  b[1] = 5;
  b[2] = 6;
}
```

You can change the original array through b.

Array Parameters: The two methods are completely equivalent

```
// array parameter
void sub(int b[], int size)
{
    b[0] = 4;
    b[1] = 5;
    b[2] = 6;
}
```

Array names and pointers are fundamentally the same.

Accessing elements using array notation

In a function parameter, `int b[]` is the same as `int *b`.

```
// pointer parameter
void sub(int *b, int size)
{
    *b = 4;
    *(b+1) = 5;
    *(b+2) = 6;
}
```

Accessing elements using pointer notation

Example:

If `b` points to the first element, `b = &arr[0]`

Then:

`*b` → `arr[0]`, `*(b+1)` → `arr[1]`, `*(b+2)` → `arr[2]`

Array Parameters with Functions

```c
void sub( int b [], int n );

int main( void )
{
    int a[3] = { 1,2,3 };
    printf( "%d %d %d\n" , a[0], a[1], a[2]);
    sub(a, 3);
    printf( "%d %d %d\n" , a[0], a[1], a[2]); // changed!
    return 0;
}

void sub( int b [], int n )
{
    b[0] = 4; b[1] = 5; b[2] = 6;
}
```

Advantages of Using Pointers

- Pointers are faster than index notation, since there is no need to convert an index into an **address** (though modern compiler optimization narrows this gap).
- You can create advanced data structures such as linked lists and binary trees.
- Call by reference: you can change the value of a variable outside the function by using a **pointer** as a parameter.

Cautions When Using Pointers

- Pointers are both a strength and a weakness of the C language
  - developers must use them responsibly.
- Common bug source: an overflow can occur when converting between data types of different sizes through a **pointer**.
- Always double-check: is this **pointer** initialized? Does it still point somewhere valid?

Summary

- A **pointer** stores the **address** of another variable: & gets an **address**, * dereferences it.
- Pointers let a function modify the caller's variables (call by reference).
- **Pointer arithmetic** moves by the size of the pointed-to type.
- Arrays and pointers are deeply connected: a[i] is the **same** as *(a + i).

Q & A

Lab & Quiz: right after this - see you in the practice session!